Smartclock Project Resume

A beginner-friendly guide to the current firmware, project files, and next steps

Project and course requirements

The ENG4098 brief asks for a standalone microcontroller prototype that uses one or more inputs and controls one or more outputs. It does not specify C or C++; this project uses C++ because that is the language used by the Arduino Due framework and its libraries.

The brief also asks the group report to identify tutorials, guides, or AI tools used, including how they were used. Keep a note of outside help as the project develops.

Project files

- **src/main.cpp** - the firmware: startup, component setup, encoder/button handling, sensor reads, and display updates.
- **include/pins.h** - named Arduino Due pin assignments.
- **platformio.ini** - board and library configuration for PlatformIO.
- **docs/code-guide.md** - a plain-language explanation of the code and basic C++ terms.
- **docs/hardware.md** - wiring, power rails, pin map, and hardware cautions.
- **README.md** - project overview and build instructions.

The empty examples/ and lib/ placeholder folders were removed. The .pio/ folder is generated build output and is ignored by Git.

What the firmware currently does

- Starts the DS3231 RTC, DHT11 sensor, DFPlayer Mini, ST7735S TFT, rotary encoder, and motor output.
- Displays time/date, temperature, humidity, audio state and volume, and motor state.
- Encoder rotation changes DFPlayer volume, limited to 0-30; startup volume is 10.
- A short encoder-button press toggles play/pause for track 1. Holding it for at least 0.8 seconds and releasing toggles the motor.
- Reads the DHT11 about every 2 seconds and refreshes the screen about once per second.
- Initializes six LED outputs off. No LED pattern, alarm schedule, or other custom clock behavior has been chosen yet.
- If the RTC reports that it lost power, sets it to the firmware build date/time. This is a starter behavior, not a way to set the correct time interactively.

Hardware details to remember

- Battery: two Li-ion cells in series, 7.4 V nominal and up to 8.4 V fully charged. The battery powers Due VIN and the LM2596 input.

- The Due 3.3 V rail and LM2596 3.3 V output are separate rails. Per the clarified wiring, the latter powers the motor, DFPlayer, and DHT11. Grounds must be common.
- Motor transistor control is on Due D9, with PWM. The current duty setting is 200/255. The motor itself is powered from the LM2596 rail, not from the Due pin.
- The TFT is listed as a 1.8-inch, 128x160 SPI display using the ST7735S controller. The project uses Adafruit_ST7735 and Adafruit_GFX.
- TFT backlight is on D28, which does not provide hardware PWM on the Due, so it is currently set on continuously.
- LED1 uses D1, which is also the Due programming-port UART TX pin. If uploads fail, move LED1 and update the pin list.
- The Due has 3.3 V GPIO. Do not apply 5 V to its input pins. Verify the battery pack, 2-cell charger/protection, supply rails, motor diode, and transistor ratings with your course/lab guidance before powering.

Build and Git status

The latest checked command, **py -m platformio run**, completed successfully for the Arduino Due. It reported two non-fatal compiler warnings about the time/date text buffer sizes in the current sketch. The firmware has not been tested on the physical hardware.

The project changes are local and uncommitted; nothing has been committed or pushed. Git is the version-history tool, while GitHub is the online hosting service. A commit saves a named checkpoint locally; pushing sends committed work to GitHub.

A step-by-step workflow

- First, describe one feature in everyday language: what starts it, what should happen, and how it stops.
- Translate that behavior into a tiny flowchart or numbered steps before writing code.
- Change only the relevant small part of the program. Understand what each changed line does.
- Build with **py -m platformio run**, then test that one behavior on the hardware.
- Review the change together, then make a Git commit as a checkpoint when it is understood and working.

Next learning step: choose the first clock behavior you want to implement, then work through its flow and code one small change at a time.